import streamlit as

```
import streamlit as st
import pandas as pd
import re
from pathlib import Path
from datetime import datetime

st.set_page_config(
    page_title="NIDS Security Dashboard",
    page_icon="🛡️",
    layout="wide"
)

LOG_FILE =
Path(__file__).with_name("nids_alerts.log")

st.title("🛡️ Network Intrusion Detection
System")
st.caption("Local network security
monitoring dashboard")
```

```
if st.button("Refresh Dashboard"):
    st.rerun()

st.divider()

# Read alerts from the NIDS log file
if LOG_FILE.exists():
    lines = LOG_FILE.read_text(
        encoding="utf-8",
        errors="replace"
    ).splitlines()
else:
    lines = []

records = []

for line in lines:
    match = re.search(
        r"\[(.*?)\]\s+ALERT:\s+(.*)",
        line
    )
```

```
if not match:
    continue

timestamp, message = match.groups()

ip_match = re.search(
    r"Source:\s*((?:\d{1,3}\.){3}\d{1,3})",
    message
)

if "Port Scan" in message:
    alert_type = "Possible Port Scan"
elif "High Traffic" in message:
    alert_type = "High Traffic"
else:
    alert_type = "Other"

records.append({
    "Time": timestamp,
    "Alert Type": alert_type,
    "Source IP": ip_match.group(1) if
```

```
ip_match else "Unknown",
        "Details": message
    })

df = pd.DataFrame(
    records,
    columns=["Time", "Alert Type", "Source IP",
"Details"]
)

col1, col2, col3 = st.columns(3)

col1.metric("Total Alerts", len(df))

if not df.empty:
    col2.metric(
        "Port Scan Alerts",
        int((df["Alert Type"] == "Possible Port
Scan").sum())
    )
    col3.metric(
        "High Traffic Alerts",
```

```
        int((df["Alert Type"] == "High
Traffic").sum())
    )
else:
    col2.metric("Port Scan Alerts", 0)
    col3.metric("High Traffic Alerts", 0)

st.divider()
st.subheader("🚨 Security Alert History")

if df.empty:
    st.info(
        "No alerts recorded yet. Keep NIDS
running "
        "and refresh this dashboard after an
alert."
    )
else:
    st.dataframe(
        df.iloc[::-1],
        use_container_width=True,
        hide_index=True
```

```
    )

    st.subheader("Alert Types")
    st.bar_chart(df["Alert
Type"].value_counts())

    st.subheader("Top Alerting Source IPs")
    st.bar_chart(df["Source
IP"].value_counts().head(10))

    st.download_button(
        "Download Alert Report (CSV)",

data=df.to_csv(index=False).encode("utf-8
"),
        file_name="nids_alert_report.csv",
        mime="text/csv"
    )

st.caption(
    "Educational prototype. Alerts are
rule-based and "
```

"may include false positives. Refresh to load new logs."
)